

Hair Cutting Robots

Lecture 4: Hotaru Framework Basics

Supervisor - Prof. Dr. Mohammed Aquil Mirza, COMP

Student Mentors - Mr. Ruan Haozhe, Mr. Jiang Suyu, Mr. Jiuyuan Zhao

May 2026

The Hong Kong Polytechnic University

Group Division

Group ID	Topic	Group ID	Topic
1	D3	2	D5
3	W3	4	W2
5	D1	6	W5
7	W1	8	D4
9	W4	10	D2



Minimal Runnable Application

Install the Hotaru CLI

Install

```
cargo install hotaru
```

This downloads, compiles, and installs the `hotaru` command-line tool.

Verify

```
hotaru version
```

Expected output

```
Hotaru version: 0.8.2
```

Scaffold a Web Project

Create the project

```
hotaru new web_hello  
cd web_hello
```

The CLI lays down a complete, ready-to-run project. We will look at the files next.

What Was Generated

The project tree

```
web_hello/  
  Cargo.toml      <- what to build  
  build.rs        <- setup script (do not touch)  
  src/  
    main.rs       <- the actual program  
  templates/      <- HTML pages (empty for now)  
  programfiles/   <- static files: CSS, JS, images
```

We will focus on `Cargo.toml` and `src/main.rs`. Everything else is scaffolding.

Cargo.toml

```
[package]
name = "web_hello"
version = "0.1.0"
edition = "2024"
build = "build.rs"

[dependencies]
hotaru = "0.8.1"
tokio = { version = "1", features = ["full"] }
```

Top half — project identity. The build line points to a setup script.

Bottom half — two dependencies: hotaru (the framework) and tokio (the async engine). "full" enables every tokio feature — the simplest choice while learning.

Skip the Plumbing

`build.rs`

Runs before the program compiles. Copies `templates/` and `programfiles/` next to the binary so the program can find them at runtime.

`src/resource.rs`

Auto-written by `build.rs` on every build. Provides a helper for locating those files at runtime.

You will never edit either of these.

Treat them as scaffolding the CLI put in place.

Run It

Start the server

```
cargo run
```

Expected console output

```
Starting Hotaru server
```

Visit the page

Open a browser and go to:

```
http://127.0.0.1:3003/
```

It Works

The browser shows:

Hello, world!

You just wrote a web server in Rust.

We have not explained *how* yet. That is the job of the next two sections.



The Four Roles

Two Sides of Every Network Conversation

Every network conversation has two participants.

One side waits. It sits there with a port open, ready to react to whoever shows up. We call it the *server*.

The other side starts the conversation. It picks a server, sends a request, and reads back a reply. We call it the *client*.

That one distinction — who waits, who starts — is the only axis we need today.

The Server Side: Inbound

A server's job is to react.

Request flows in. Some client out there says “please give me this”.

Response flows out. The server decides what to send back.

The server itself does not pick when this happens. It is woken up by every incoming request and goes back to sleep when it is done.

The Client Side: Outbound

A client's job is to act.

Request flows out. The client decides to ask for something and dials a server.

Response flows in. The server answers; the client reads the answer.

The client picks the moment, the destination, and the question. The server has no say.

Endpoints Live on the Server

A *server* is the box. An *endpoint* is one of the handlers inside it.

What an endpoint is

A URL paired with a function. When an incoming request matches that URL, the function runs and its return value becomes the reply.

A server is a collection of endpoints

One for the home page, one for the login page, one for an upload form, and so on. Today we have just one.

Outpoints Live on the Client

The mirror image. A *client* is the box. An *outpoint* is one of the handlers inside it.

What an outpoint is

A named outgoing call the client knows how to make. “Call this URL with these arguments; here is what to do with the reply.”

We will not write one today.

The shape is the same as an endpoint, with the arrows flipped.

The Four Roles in One Picture

	who starts the talk	who handles
Server	the other side	endpoints
Client	this side	outpoints

Server / Client — the box that is running.

Endpoint / Outpoint — the handlers living inside that box.

Today We Touch All Four

By the end of this lecture, our program will use every role on the map.

First example: `web_hello`

A Server with one Endpoint. Just answers. Top-left only.

Second example: `time_proxy`

The same Server, plus a Client and an Outpoint. When a browser asks our server for the time, the server in turn asks `timeapi.io` and forwards the answer.

The Idea That Makes Hotaru Hotaru

In most frameworks

You write the handler in one place, then write a second line that says “please add this handler to the server’s route table”. Forget that line and the handler is silently invisible.

In Hotaru

You write the handler. That is it. It attaches itself to the server when the program starts.

Why this matters

Handlers can live in any file. There is no central router file to maintain. Adding a new endpoint is one block in one file, not two.

The Five Pieces of `main.rs`

We have already seen them on screen. The next section walks through each one in detail.

1. `use` lines — bring names into scope.
2. `LServer!{ APP = ... }` — declare the server.
3. `#[tokio::main] async fn main` — start the engine and run the server.
4. `endpoint!{ APP.url("/"), ... }` — declare one endpoint, auto-attached to `APP`.
5. `text_response(...)` — what the endpoint returns.



The Heart of the App: `src/main.rs`

The Whole File

```
use hotaru::prelude::*;
use hotaru::http::*;
#[tokio::main]
async fn main() { APP.clone().run().await; }

LServer!(
    APP = Server::new()
        .binding("127.0.0.1:3003")
        .single_protocol(ProtocolBuilder::new(HTTP::server(HttpSafety::default()))))
        .build()
);
endpoint!{
    APP.url("/"),
    /// Hello world function
    pub hello_world <HTTP> { text_response("Hello, world!") }
}
```

The use Lines

```
use hotaru::prelude::*;  
use hotaru::http::*;
```

`hotaru::prelude`

Re-exports the items you almost always need: `Server`, `ProtocolBuilder`, the `LServer!` and `endpoint!` macros, and so on.

`hotaru::http`

HTTP-specific items: the `HTTP` type, `HttpSafety`, helpers like `text_response`.

`#[tokio::main]` — Making `main` Async

```
#[tokio::main]
async fn main() {
    APP.clone().run().await;
}
```

The problem

Rust's `main` cannot be `async` on its own. Something has to start an `async` runtime first.

The fix

`#[tokio::main]` rewrites our `async main` at compile time into a normal `main` that starts the `tokio` runtime and runs our code on it. “Please wrap me so I can use `.await`.”

Inside main: `APP.clone().run().await;`

`APP.clone().run().await;`

APP

A global handle to our server, defined a few lines below by the `LServer!` macro.

`.clone()`

APP is shared, reference-counted state. `clone()` hands us a cheap copy of the handle, not a copy of the whole server.

`.run().await`

`run()` returns a future that listens for connections forever. `.await` hands control to the runtime and only returns when the server stops.

The LServer! Macro

```
LServer!(  
    APP = Server::new()  
        .binding("127.0.0.1:3003")  
        .single_protocol(...)  
        .build()  
);
```

Declares a lazily-initialised global called APP. The first time someone touches APP, the expression on the right runs; the result is cached forever.

A macro (and not a normal let) because APP must be visible to both main and the endpoint! macro that registers handlers on it.

Building the Server: the Builder Pattern

```
Server::new()  
    .binding("127.0.0.1:3003")  
    .single_protocol(...)  
    .build()
```

What this is

The *builder pattern*. You start with `Server::new()`, chain configuration methods that each return the builder again, and finish with `.build()` which produces the finished `Server`.

Why this style

It reads top to bottom like a recipe, every option is named, and you cannot accidentally forget a step — `.build()` will refuse to compile if a required setting is missing.

```
.binding("127.0.0.1:3003")
```

Address

`127.0.0.1` is *localhost* — only your own machine can reach it. Use `0.0.0.0` when you want to accept connections from the network.

Port

`3003` is the TCP port. Ports below 1024 need administrator privileges, so development servers usually pick something in the thousands.

Result

Once running, the server is reachable at `http://127.0.0.1:3003` from a browser on the same computer.

```
.single_protocol(...)  
  
.single_protocol(  
    ProtocolBuilder::new(HTTP::server(HttpSafety::default()))  
)
```

Why “single”

A Hotaru server can speak more than one protocol at once. `single_protocol` is the shortcut for the one-protocol case.

What is wrapped

`HTTP::server(...)` picks HTTP in server mode. `HttpSafety::default()` provides sensible request-size limits. `ProtocolBuilder::new(...)` wraps the result so the Server builder accepts it.

The endpoint! Macro

```
endpoint!{  
  APP.url("/"),  
  pub hello_world <HTTP> {  
    text_response("Hello, world!")  
  }  
}
```

Registers a route on APP: when a request matches the URL, the body runs and its return value becomes the response.

A macro because routing involves type-level wiring (URL matching, protocol selection, handler signature). The macro hides all of it.

Anatomy of the endpoint! Block

```
APP.url("/")
```

Attaches the handler to APP at path `/`.

```
pub hello_world <HTTP>
```

The handler's name and the protocol it serves. `<HTTP>` tells the macro to generate the right signature for HTTP.

```
text_response("Hello, world!")
```

A helper from `hotaru::http` that builds a plain-text 200 OK response with the given body.

Putting It All Together

What happens when you run `cargo run`

1. `build.rs` runs, copying assets and generating `resource.rs`.
2. The crate compiles.
3. `main` starts the tokio runtime.
4. `APP` is built lazily on first use.
5. `APP.run()` starts listening on `127.0.0.1:3003`.
6. A browser request to `/` dispatches into `hello_world`.
7. `text_response("Hello, world!")` is sent back.

That is the entire request-response cycle. Every frame from here on will add one piece on top of this skeleton.



A Server That Also Asks: `time_proxy`

The Next Step

`web_hello` is a program that only *answers*. `time_proxy` is a program that also *asks*.

The goal

When a browser hits our server, the server in turn fetches the current UTC time from a public API (`timeapi.io`) and forwards the JSON to the browser.

The data path

browser → our Server (HTTP) → our Client (HTTPS) → `timeapi.io` → JSON → browser

Cargo.toml: Turn On HTTPS

`timeapi.io` only serves HTTPS, so we need TLS support.

Before

```
hotaru = "0.8.2"
```

After

```
hotaru = { version = "0.8.2", features = ["https"] }
```

The `https` feature pulls in `HTTPS`, `TlsTransport`, `TlsOutboundTarget`, and `TlsClientConfig`.

The Whole File

```
use hotaru::http::*;
use hotaru::prelude::*;
#[tokio::main]
async fn main() { APP.clone().run().await; }

LServer!(
    APP = Server::new()
        .binding("127.0.0.1:3003")
        .single_protocol(ProtocolBuilder::new(HTTP::server(HttpSafety::default()))))
        .build()
);
pub static TIME_CLIENT: Lazy<Arc<Client<TlsTransport>>> = Lazy::new(|| {
    Client::<TlsTransport>::new()
        .target(TlsOutboundTarget::new("timeapi.io", 443, TlsClientConfig::default()))
        .single_protocol(ProtocolBuilder::new(HTTPS::client(HttpSafety::default()))))
        .build()
});
```

The Whole File (cont'd)

```

outpoint! {
  TIME_CLIENT.url("/api/v1/time/current/utc"),
  fetch_utc_time <HTTPS> {
    send;
    Ok(req)
  }
}

endpoint! {
  APP.url("/"),
  pub index <HTTP> {
    let mut outbound = request_templates::get_request("/api/v1/time/current/utc");
    outbound.meta.set_host(Some("timeapi.io".to_string()));
    match run!(TIME_CLIENT<HTTPS>::fetch_utc_time, outbound).await {
      Ok(Ok(resp)) => /* forward resp body */,
      Ok(Err(e))   => /* upstream error   -> 502 */,
      Err(e)       => /* lookup error     -> 500 */,
    }
  }
}

```

A Second Box: the Client

The Server piece (`LServer! APP = ...`) is unchanged from `web_hello`. The new piece is a second top-level static:

```
pub static TIME_CLIENT: Lazy<Arc<Client<TlsTransport>>> = Lazy::new(|| {
    Client::<TlsTransport>::new()
        .target(TlsOutboundTarget::new("timeapi.io", 443, TlsClientConfig::default()))
        .single_protocol(ProtocolBuilder::new(HTTPS::client(HttpSafety::default())))
        .build()
});
```

`.target(...)` — where this client dials. Hostname, port, TLS settings.

`HTTPS::client(...)` — the same protocol object as `HTTP::server`, but in client mode and TLS-wrapped.

Why Not LClient!?

For the Server we used LServer!. The symmetric macro LClient! exists, but we did not use it. Why?

The macro hardcodes the type

```
pub static NAME: SClient = Lazy::new(|| EXPR);
```

And SClient defaults to Lazy<Arc<Client<TcpTransport>>>. There is no syntax to ask LClient! for a TLS-transport client.

The workaround

Declare the static directly so we can pin TS = TlsTransport. This is the only place we step outside the macros.

The Outpoint

```
outpoint! {  
  TIME_CLIENT.url("/api/v1/time/current/utc"),  
  fetch_utc_time <HTTPS> {  
    send;  
    Ok(req)  
  }  
}
```

Same shape as `endpoint!` but flipped: attaches to a *Client*, runs an *outgoing* call.

`send;` is recognised by the `outpoint!` macro. It fires the request on the wire. After this line, `req` holds the response.

Building the Outgoing Request

The endpoint body builds the request that `run!` will hand to the outpoint:

```
let mut outbound = request_templates::get_request(  
    "/api/v1/time/current/utc"  
);  
outbound.meta.set_host(Some("timeapi.io".to_string()));
```

`request_templates::get_request(path)`

Creates an `HttpRequest` with method `GET` and the given path.

`set_host(...)`

Sets the HTTP Host header. `nginx` rejects requests without it.

run!: One Line to Fire

```
run!(TIME_CLIENT<HTTPS>::fetch_utc_time, outbound).await
```

Reads as

“On client `TIME_CLIENT`, run the outpoint named `fetch_utc_time`, speaking `HTTPS`, with this outbound request.”

Returns

```
Result<Result<HttpResponse, ProtocolError>, UrlError>
```

A two-layer Result — explained next.

Three Outcomes

`Ok(Ok(resp))` — happy path. Wire succeeded; we have an `HttpResponse`.

`Ok(Err(e))` — the outpoint was found but the call itself failed. TLS handshake, timeout, malformed reply, etc. We answer the browser with 502 Bad Gateway.

`Err(e)` — the outpoint name was not registered (typo or missing protocol). We answer with 500 Internal Server Error.

Two layers because two things can go wrong independently: *finding* the call and *making* the call.

Forwarding the Body

```
let body_bytes: Vec<u8> = match resp.body {  
    HttpBody::Text(s)           => s.into_bytes(),  
    HttpBody::Binary(b)        => b,  
    HttpBody::Buffer { data, .. } => data,  
    -                           => Vec::new(),  
};  
let body_str = String::from_utf8_lossy(&body_bytes).to_string();  
text_response(body_str)
```

Why a match

`HttpBody` is an enum — it can hold a string, raw bytes, or a buffered chunk. We do not know which one the upstream sent, so we unify them into `Vec<u8>` and decode as UTF-8.

The Host Header: a Temporary Wart

The `set_host` line is **not** something you should have to write.

What *should* happen

The client knows its target hostname. The framework should fill the Host header automatically when missing.

What *does* happen

The auto-fill logic reads from a field the Client never populates from its target. Host ends up empty, nginx says 400, and we work around it. A fix is in flight; once it lands, the `set_host` line goes away.

Recap: All Four Roles in 30 Lines

1. Browser hits our **Server** at `/`.
2. The Endpoint at `/` builds an outbound request and calls `run!` on the **Client**.
3. `run!` dispatches into the **Outpoint** `fetch_utc_time`.
4. `send`; performs the TLS handshake and the HTTPS GET to `timeapi.io`.
5. Upstream replies with JSON. `req.response` now holds it.
6. Back in the Endpoint, we extract the body and return it.
7. Browser sees `{"utc_time": "..."}.`



Hotaru MQTT: One-to-Many on the Same Engine

From One-to-One to One-to-Many

HTTP is a two-party conversation: one client asks, one server answers, then they hang up.

HCR is not shaped like that.

When the sandbox stamps a plan *approve*, that single message needs to reach *everyone who cares*: the desktop viewport, the browser viewport, the audit log, eventually the real robot. Same message, many readers.

The shape we need is a *bus* — one publisher, many subscribers, decoupled in time. That is exactly the niche MQTT was designed for.

The Fifth Role: the Broker

The four-role map from §2 had two boxes (Server, Client) and two handlers (Endpoint, Outpoint). MQTT adds one more box:

Server	waits, answers (HTTP-style)
Client	dials, asks
Broker	<i>relays</i> — accepts publishes, fans them out to matching subscribers

A Broker is a Server with extra state: a table of *who is subscribed to what*. It does not answer questions of its own — it forwards.

In Hotaru, the Broker is a normal Server running the MQTT protocol, plus a Broker struct living in runtime statics.

Where HCR Uses It

From the handbook (§2.3): `hotaru_mqtt` is the shared bus on both tracks.

Desktop track (D5 Bus)

The initiator process *embeds* the broker. No central server. Peers connect locally, the broker fans out twin state / sandbox decisions to all of them.

Web track (W5 Bus)

The broker runs on a dedicated server. Browsers connect over MQTT-over-WSS. Same topic graph, same schemas.

What is shared (the contract): the topic strings, the message schemas, the five worked-example fixtures. Everything else — transport, GPU, UI — is allowed to diverge.

Cargo.toml

hotaru_mqtt is in-house and not on crates.io. Pull it in as a path dependency:

```
[dependencies]
hotaru          = "0.8.2"
hotaru_mqtt     = { path = "../hotaru_mqtt" }
tokio           = { version = "1", features = ["full"] }
bytes           = "1"
```

bytes shows up because MQTT payloads are Bytes — a reference-counted byte slice. Fanning a 4 KB approve message to 20 subscribers is 20 Arc bumps, not 20 memcpy's.

The Server Is the Same Shape

`web_hello` ran HTTP:

```
.single_protocol(ProtocolBuilder::new(
  HTTP::server(HttpSafety::default()))))
```

An MQTT broker swaps one line:

```
.single_protocol(ProtocolBuilder::new(
  MQTT::server()))
```

That is the entire transport swap. `LServer!`, `.binding(...)`, `.build()`, `APP.clone().run().await` — unchanged.

The interesting work is on *the layer above*: who subscribes, who publishes, what the topics

Registering the Broker

A Server alone is not a broker — it has no subscription table. We register the Broker struct into runtime statics so the MQTT handler can find it:

```
use hotaru_mqtt::{Broker, BROKER_STATICS_KEY, MQTT};

let broker = Broker::<TcpStream>::new();

let runtime = AppBuilder::new()
    .set_static(BROKER_STATICS_KEY, broker.clone())
    .build();
```

The `broker.clone()` we keep is a cheap Arc handle. **Anyone with that handle can publish into the bus** — including code that is not even speaking MQTT.

A Subscriber Endpoint

The same endpoint! macro from §3, speaking MQTT:

```
endpoint! {
  APP.url("twin/<robot_id>/state"),
  pub on_twin_state <MQTT> {
    let id = req.param("robot_id").unwrap();
    let payload = req.incoming.as_ref().unwrap().payload();
    update_viewport(id, payload).await;
    Ok(req)
  }
}
```

Same five pieces: attach to APP, give a path, name the handler, declare <MQTT>, write the body.

The macro registers the path with the broker so any PUBLISH to twin/r1/state (or any

Topic Grammar: HTTP Path, MQTT Wire

You write paths in the same HTTP grammar as the rest of Hotaru. The framework translates to MQTT wire syntax when it sends SUBSCRIBE.

	Hotaru macro path	MQTT wire filter
single segment	<id>	+
remaining path	<**rest>	#
literal	twin/r1/state	twin/r1/state

Examples

sensors/<id>/temp	wire: sensors/+/temp
alerts/<**path>	wire: alerts/#
\$SYS/broker/clients	wire: \$SYS/broker/clients

Publishing: Same run! You Already Know

The outpoint shape from §4 generalises. Same run!, MQTT request type:

```
outpoint! {
  APP.url(""),
  publish_state <MQTT> { send; Ok(req) }
}
```

```
run!(APP<MQTT>::publish_state,
  MqttRequest::Publish(PublishRequest {
    topic:   Arc::from("twin/r1/state"),
    payload: Bytes::from(serde_json::to_vec(&state)?),
    qos:     QoS::AtLeastOnce,
    retain:  false,
  })).await?;
```

The Server doubles as a Client into its own broker. The broker does the fanout: subscribers'

QoS in Robot-Speak

MQTT defines three quality-of-service levels. In HCR they map cleanly to message kinds:

QoS 0 — “at most once”

Fire and forget. Use for high-rate telemetry: hair-vertex stream, twin pose at 60 Hz. Losing a frame is fine.

QoS 1 — “at least once”

Acknowledged. May arrive twice. Use for commands: *move tool to here*. The receiver de-dups by intent.

QoS 2 — “exactly once”

Full four-way handshake (PUBLISH / PUBREC / PUBREL / PUBCOMP). Slow but unique. Use only for *approve* — the message that authorises the robot to cut. You do not want it

The Data Flow End-to-End

1. CONNECT

client sends `Connect{client_id, clean_session, keep_alive, will?}`
broker authenticates, replies `Connack{Accepted}`

2. SUBSCRIBE

client sends `Subscribe{packet_id, [(filter, max_qos)]}`
broker registers `(client_id, filter, qos)`, replies `Suback`

3. PUBLISH

publisher sends `Publish{topic, payload, qos, retain}`
broker walks its subscription table, finds matches
for each match: `cmd_tx.send(WriteCmd::Publish(...))`
each subscriber's writer actor serialises onto its own wire

4. ACK (only QoS $\geq$ 1)

One PUBLISH, Many Wires

Inside the broker, one inbound PUBLISH becomes one Bytes payload shared by every fanout. No copies.

```
broker.publish(source, packet):  
  for sub in matching_subscribers(packet.topic):  
    let qos = packet.qos.min(sub.max_qos);  
    let id = if qos > 0 { Some(sub.next_packet_id()) }  
              else { None };  
    sub.channel.cmd_tx.send(WriteCmd::Publish(  
      PublishPacket { qos, packet_id: id,  
        payload: packet.payload.clone(), // Arc bump  
        ..packet.clone() }));
```

Self-fanout suppression: the broker skips the connection that produced the publish, so a client subscribed to its own topic does not hear its own voice.

Last Will and Testament

A *will* is a message the broker promises to publish *on behalf of* a client **if that client disconnects abruptly**.

Set at CONNECT time:

```
will = Some(WillMessage { topic, payload, qos, retain })
```

Fires when the TCP connection drops without a graceful Disconnect frame, or the keep-alive grace window ($1.5\times$) expires.

Does not fire when the client sends Disconnect cleanly.

Why this matters for HCR: a robot whose link dies must not leave its peers thinking the cut is still authorised. The will publishes *robot lost* on a known topic; subscribers react fail-closed.

AIoT Closed Loop: HTTP Talks to MQTT

The Broker handle lives in runtime statics, so *any* endpoint can publish — including HTTP:

```
endpoint! {
  APP.url("/api/approve"),
  pub approve <HTTP> {
    let broker = req.statics::<Broker<_>>(BROKER_STATICS_KEY)?;
    broker.publish(&Arc::from("http-bridge"), PublishPacket {
      topic:   Arc::from("plan/approve"),
      payload: req.body_bytes(),
      qos:     QoS::ExactlyOnce, retain: false,
      dup:     false, packet_id: None,
    }).await;
    text_response("ok")
  }
}
```

What We Reused, What We Added

Reused (no new vocabulary)

LServer!, endpoint!, outpoint!, run!, <protocol>, HTTP-style path grammar, builder pattern, tokio runtime, runtime statics.

Added (one new role + four new types)

- **Broker** — Server with a subscription table; lives in runtime statics
- **MqttRequest** — Publish / Subscribe / Unsubscribe
- **IncomingPublish** — what a subscribed endpoint reads off req
- **QoS** — pick the right one for the message kind
- **WillMessage** — fail-closed on disconnect

Recap: Five Roles in One Picture

	who initiates	handler shape
Server	the other side	endpoint!
Client	this side	outpoint!
Broker	publishers	endpoint! (per subscription)

The architectural payoff for HCR

One framework, one routing language, one config-flip to swap $\text{sim} \leftrightarrow \text{real}$. The behavior contract — topics, schemas, worked-example fixtures (Appendix A/B/C) — is what every twin pair aligns to. `hotaru_mqtt` is the piece of plumbing that makes that contract executable.